法律声明

- 本课件包括演示文稿、示例、代码、题库、视频和声音等内容，小象学院和主讲老师拥有完全知识产权的权利；只限于善意学习者在本课程使用，不得在课程范围外向任何第三方散播。任何其他人或机构不得盗版、复制、仿造其中的创意及内容，我们保留一切通过法律手段追究违反者的权利。

- 课程详情请咨询
  - 微信公众号：小象
  - 新浪微博：ChinaHadoop

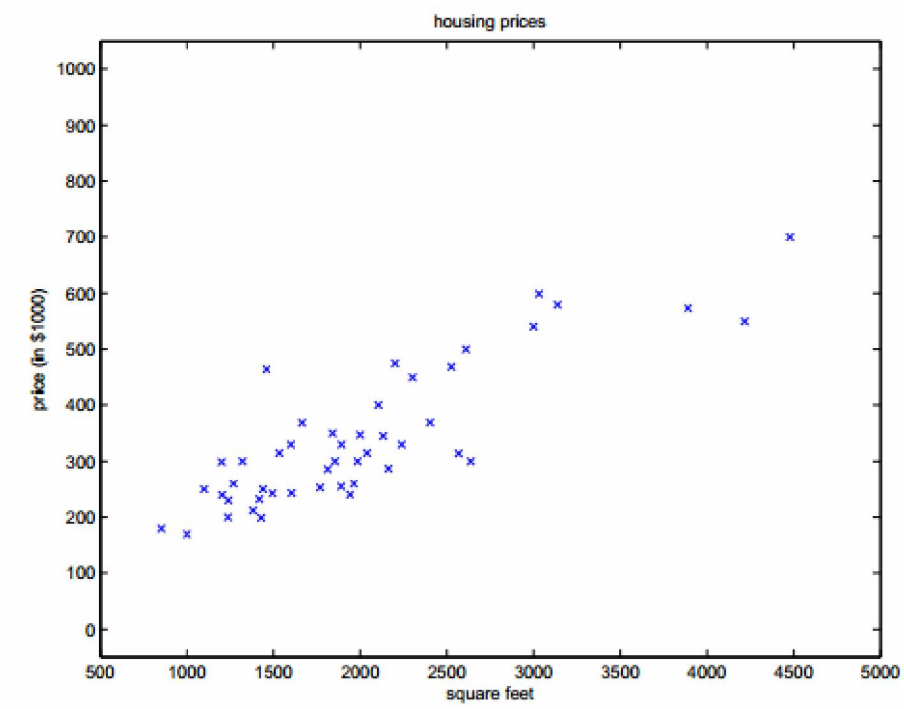
回归

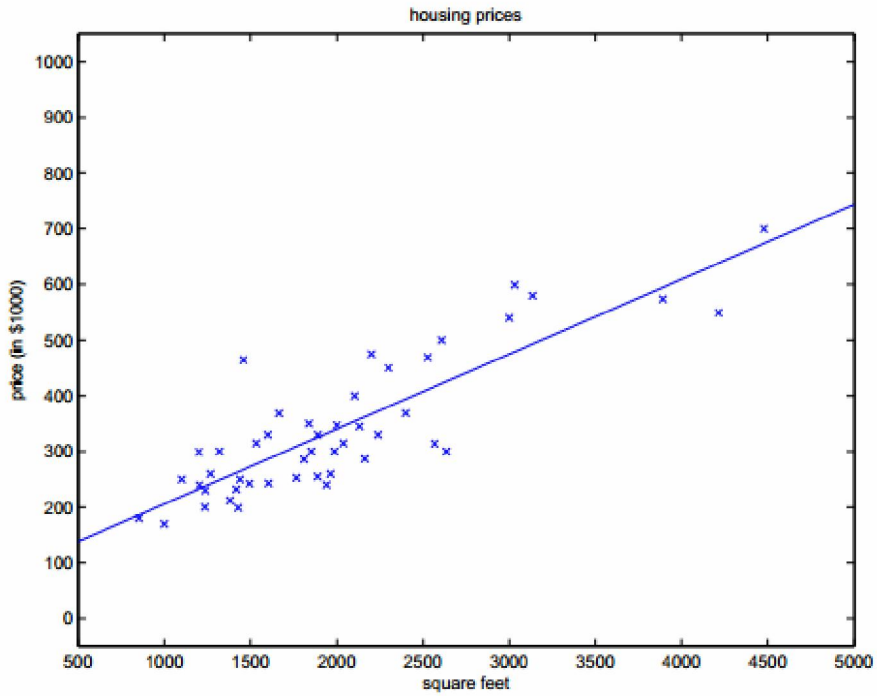
小象学院 ChinaHadoop.cn

邹博

线性回归

- y=ax+b

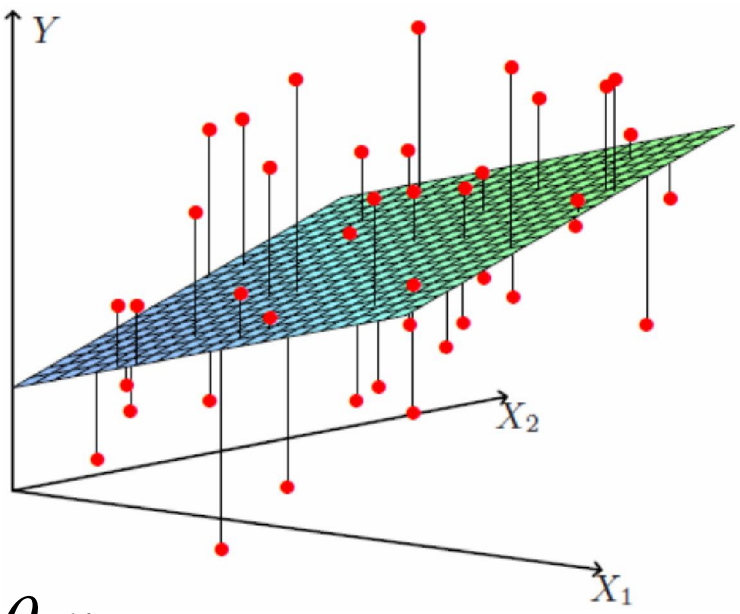

housing prices
price (in $1000)
square feet
0
100
200
300
400
500
600
700
800
900
1000
500
1000
1500
2000
2500
3000
3500
4000
4500
5000


housing prices
price (in $1000)
square feet
0
100
200
300
400
500
600
700
800
900
1000
500
1000
1500
2000
2500
3000
3500
4000
4500
5000

多个变量的情形

- 考虑两个变量

Living area (feet^2)	#bedrooms	Price (1000$s)
2104	3	400
1600	3	330
2400	3	369
1416	2	232
3000	4	540
⋮	⋮	⋮



$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h_\theta(x) = \sum_{i=0}^{n} \theta_i x_i = \theta^T x$$

使用极大似然估计解释最小二乘

$$y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$$

the $\epsilon^{(i)}$ are distributed IID (independently and identically distributed) according to a Gaussian distribution (also called a Normal distribution) with mean zero and some variance σ^2

- 误差 $\varepsilon^{(i)}(1 \leqslant i \leqslant m)$是独立同分布的，服从均值为0，方差为某定值 σ^2的高斯分布。
  - 原因：中心极限定理

中心极限定理的意义

- 实际问题中，很多随机现象可以看做众多因素的独立影响的综合反应，往往近似服从正态分布。
  - 城市耗电量：大量用户的耗电量总和
  - 测量误差：许多观察不到的、微小误差的总和
  - 注：应用前提是多个随机变量的和，有些问题是乘性误差，则需要鉴别或者取对数后再使用。

似然函数 $y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right)$$

$$p(y^{(i)}|x^{(i)};\theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

$$\begin{aligned} L(\theta) &= \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)};\theta) \\ &= \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \end{aligned}$$

高斯的对数似然与最小二乘

$$
\begin{aligned}
\ell(\theta) &= \log L(\theta) \\
&= \log \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
&= \sum_{i=1}^{m} \log \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
&= m \log \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^{m} (y^{(i)} - \theta^T x^{(i)})^2.
\end{aligned}
$$

$$
J(\theta) = \frac{1}{2} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2
$$

θ的解析式的求解过程

- 将M个N维样本组成矩阵X：
  - 其中，X的每一行对应一个样本，共M个样本
  - X的每一列对应样本的一个维度，共N维
    - 其实还有额外的一维常数项，全为1
- 目标函数 $J(\theta)=\frac{1}{2}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right)-y^{(i)}\right)^2=\frac{1}{2}(X\theta-y)^T(X\theta-y)$
- 梯度：

$$\nabla_\theta J(\theta)=\nabla_\theta\left(\frac{1}{2}(X\theta-y)^T(X\theta-y)\right)=\nabla_\theta\left(\frac{1}{2}\left(\theta^T X^T-y^T\right)(X\theta-y)\right)$$

$$=\nabla_\theta\left(\frac{1}{2}\left(\theta^T X^T X\theta-\theta^T X^T y-y^T X\theta+y^T y\right)\right)$$

$$=\frac{1}{2}\left(2X^T X\theta-X^T y-\left(y^T X\right)^T\right)=X^T X\theta-X^T y\xrightarrow{\text{求驻点}}0$$

最小二乘意义下的参数最优解

- 参数的解析式

$$\theta = \left(X^T X\right)^{-1} X^T y$$

- 若X^TX不可逆或防止过拟合，增加λ扰动

$$\theta = \left(X^T X + \lambda I\right)^{-1} X^T y$$

- “简便”方法记忆结论

$$X\theta = y \Rightarrow X^T X\theta = X^T y$$

$$\Rightarrow \theta = \left(X^T X\right)^{-1} X^T y$$

加入λ扰动后

- X^TX半正定：对于任意的非零向量u

$$uX^TXu=(Xu)^TXu\xrightarrow{令v=Xu}v^Tv\geq 0$$

- 所以，对于任意的实数λ>0，$X^TX+\lambda I$ 正定，从而可逆。保证回归公式一定有意义。

$$\theta=\left(X^TX+\lambda I\right)^{-1}X^Ty$$

线性回归的复杂度惩罚因子

- 线性回归的目标函数为：

$$J(\vec{\theta})=\frac{1}{2}\sum_{i=1}^{m}\left(h_{\vec{\theta}}\left(x^{(i)}\right)-y^{(i)}\right)^{2}$$

- 将目标函数增加平方和损失：

$$J(\vec{\theta})=\frac{1}{2}\sum_{i=1}^{m}\left(h_{\vec{\theta}}\left(x^{(i)}\right)-y^{(i)}\right)^{2}+\lambda\sum_{j=1}^{n}\theta_{j}^{2}$$

- 本质即为假定参数θ服从高斯分布。

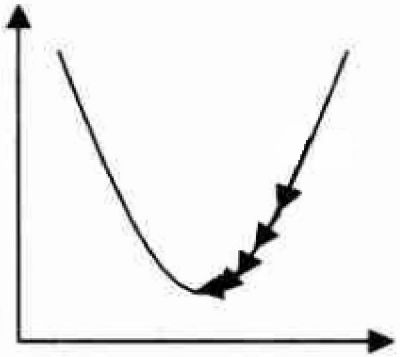
广义逆矩阵(伪逆)

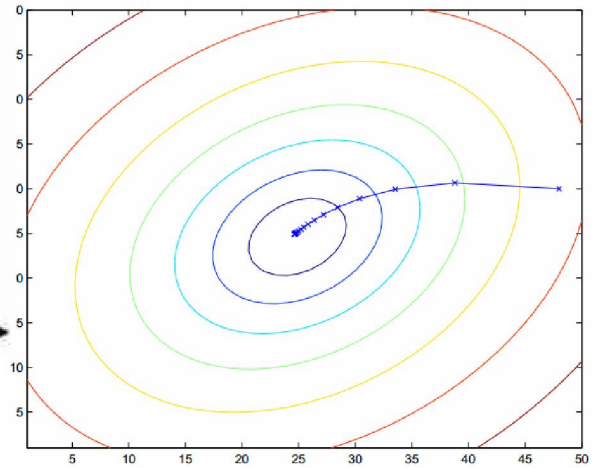
- 若A为非奇异矩阵，则线性方程组Ax=b的解为 $A^{+}=\left(A^{T}A\right)^{-1}A^{T}$
- 若A为可逆方阵，$A^{+}=\left(A^{T}A\right)^{-1}A^{T}$ 即为 A^{-1}

$$\left(A^{T}A\right)^{-1}A^{T}=A^{-1}\left(A^{T}\right)^{-1}A^{T}=A^{-1}$$

- 当A为矩阵(非方阵)时，称A^{+}为A的广义逆(伪逆)。
  - 奇异值分解SVD中继续该话题。

梯度下降算法





- 初始化θ(随机初始化)
- 迭代，新的θ能够使得J(θ)更小
- 如果J(θ)能够继续减少，返回(2)

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

- α称为学习率/步长

梯度方向

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y\right)^2 \\
&= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y\right) \\
&= \left(h_\theta(x) - y\right) \cdot \frac{\partial}{\partial \theta_j} \left(\sum_{i=0}^{n} \theta_i x_i - y \right) \\
&= \left(h_\theta(x) - y\right) x_j
\end{aligned}
$$

批处理梯度下降算法

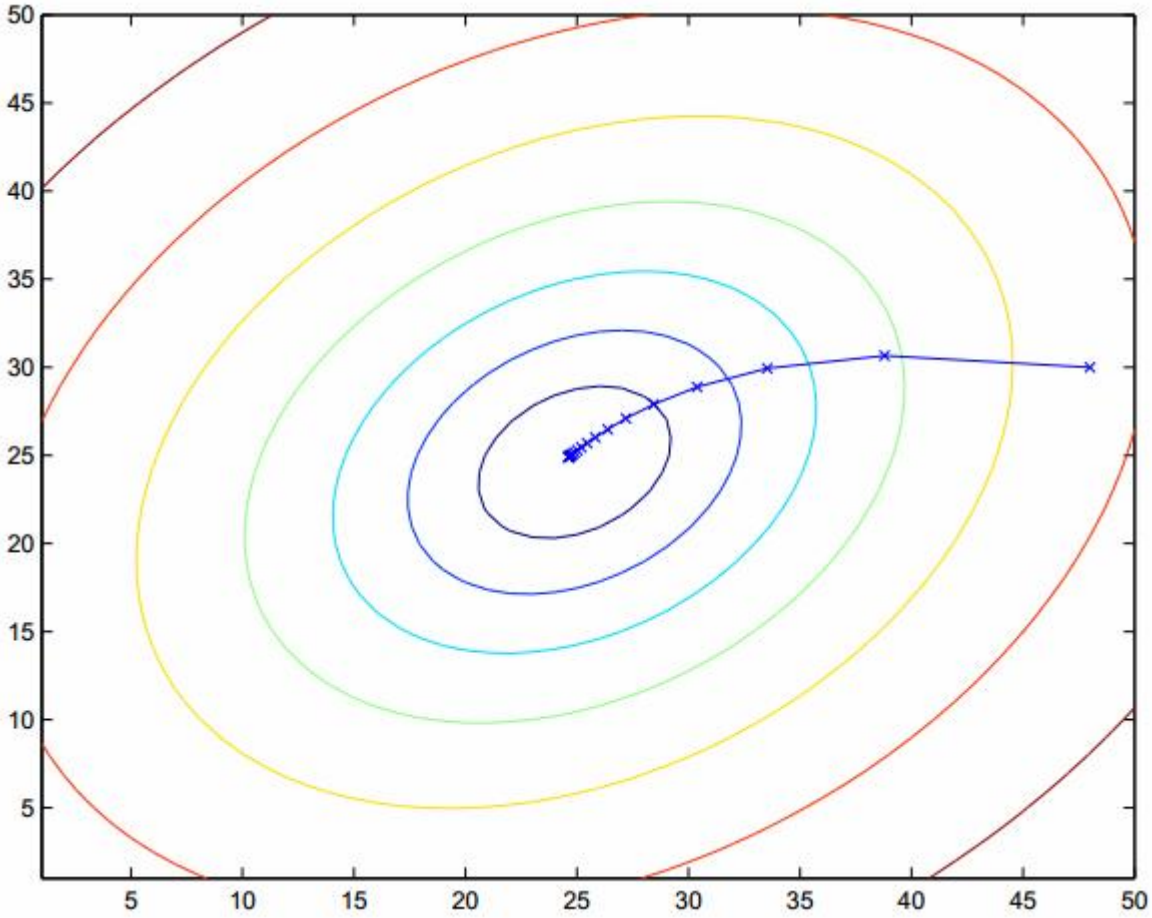
Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

gradient descent. Note that, while gradient descent can be susceptible to local minima in general, the optimization problem we have posed here for linear regression has only one global, and no other local, optima; thus gradient descent always converges (assuming the learning rate α is not too large) to the global minimum. Indeed, J is a convex quadratic function.

批处理梯度下降图示



随机梯度下降算法

```
Loop {
    for i=1 to m, {
        θ_j := θ_j + α (y^(i) − h_θ(x^(i))) x_j^(i)
    }
}
```

This algorithm is called **stochastic gradient descent** (also **incremental gradient descent**). Whereas batch gradient descent has to scan through the entire training set before taking a single step—a costly operation if m is large—stochastic gradient descent can start making progress right away, and continues to make progress with each example it looks at. Often, stochastic gradient descent gets θ "close" to the minimum much faster than batch gradient descent. (Note however that it may never "converge" to the minimum, and the parameters θ will keep oscillating around the minimum of $J(\theta)$; but in practice most of the values near the minimum will be reasonably good approximations to the true minimum.[2]) For these reasons, particularly when the training set is large, stochastic gradient descent is often preferred over batch gradient descent.

象学院
aHadoop.cn

mini-batch

- 如果不是每拿到一个样本即更改梯度，而是若干个样本的平均梯度作为更新方向，则是mini-batch梯度下降算法。

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

}

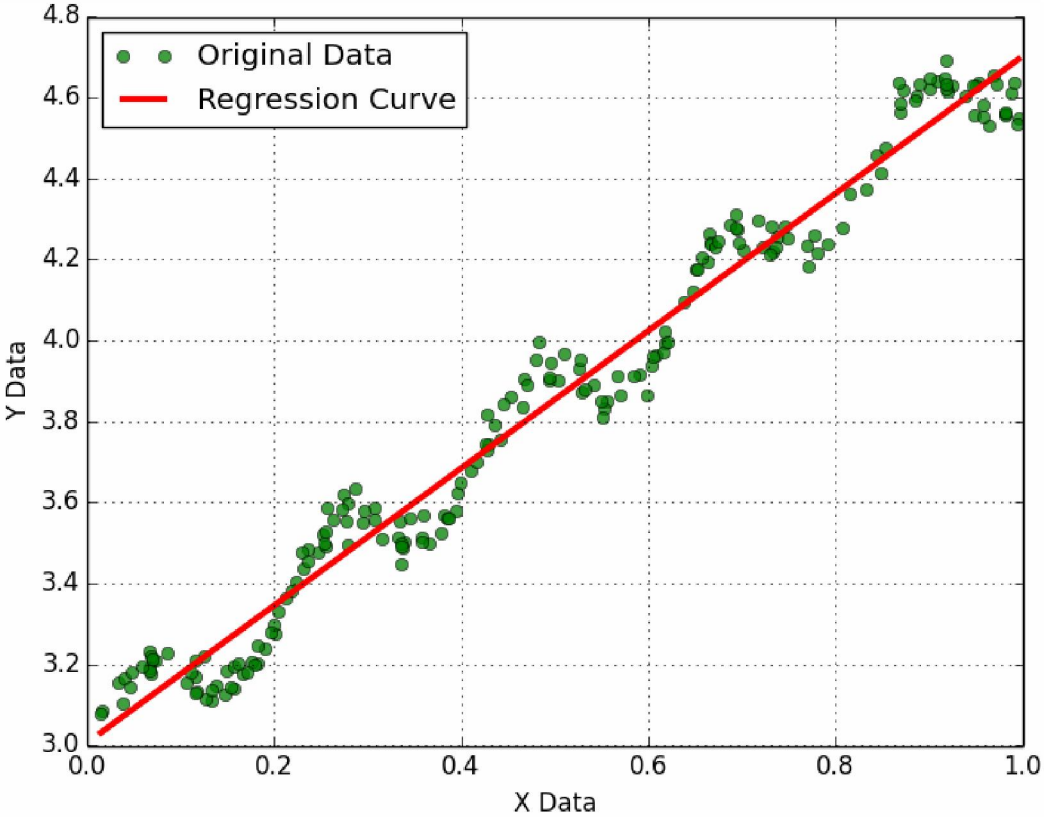
回归Code

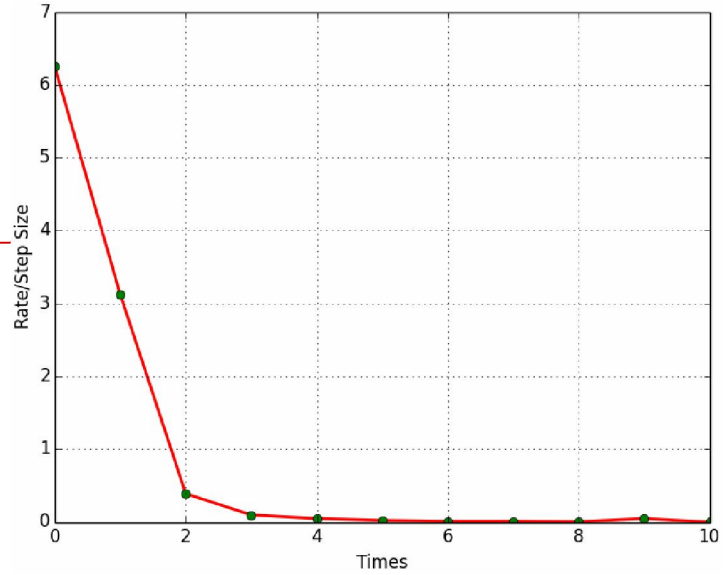
```
def calcCoefficient(data, listA, listW, listLostFunction):
    N = len(data[0])  # 维度
    w = [0 for i in range(N)]
    wNew = [0 for i in range(N)]
    g = [0 for i in range(N)]

    times = 0
    alpha = 100.0  # 学习率随意初始化
    while times < 10000:
        j = 0
        while j < N:
            g[j] = gradient(data, w, j)
            j += 1
        normalize(g)  # 正则化梯度
        alpha = calcAlpha(w, g, alpha, data)
        numberProduct(alpha, g, wNew)

        print "times,alpha,fw,w,g:\t", times, alpha, fw(w, data), w, g
        if isSame(w, wNew):
            break
        assign2(w, wNew)  # 更新权值
        times += 1

        listA.append(alpha)
        listW.append(assign(w))
        listLostFunction.append(fw(w, data))
    return w
```

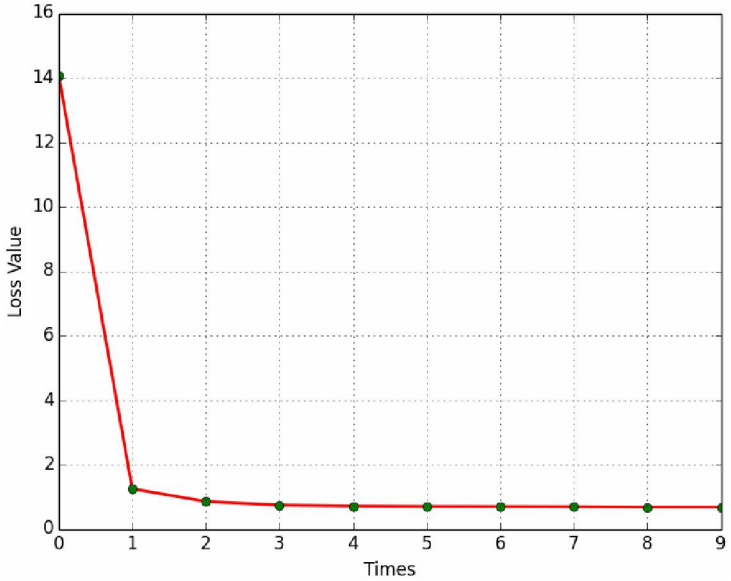
附：学习率Code

```python
# w当前值；g当前梯度方向；a当前学习率；data数据
def calcAlpha(w, g, a, data):
    c1 = 0.3
    now = fw(w, data)
    wNext = assign(w)
    numberProduct(a, g, wNext)
    next = fw(wNext, data)
    # 寻找足够大的a，使得h(a)>0
    count = 30
    while next < now:
        a *= 2
        wNext = assign(w)
        numberProduct(a, g, wNext)
        next = fw(wNext, data)
        count -= 1
        if count == 0:
            break

    # 寻找合适的学习率a
    count = 50
    while next > now - c1*a*dotProduct(g, g):
        a /= 2
        wNext = assign(w)
        numberProduct(a, g, wNext)
        next = fw(wNext, data)

        count -= 1
        if count == 0:
            break
    return a
```

线性回归、rate、Loss

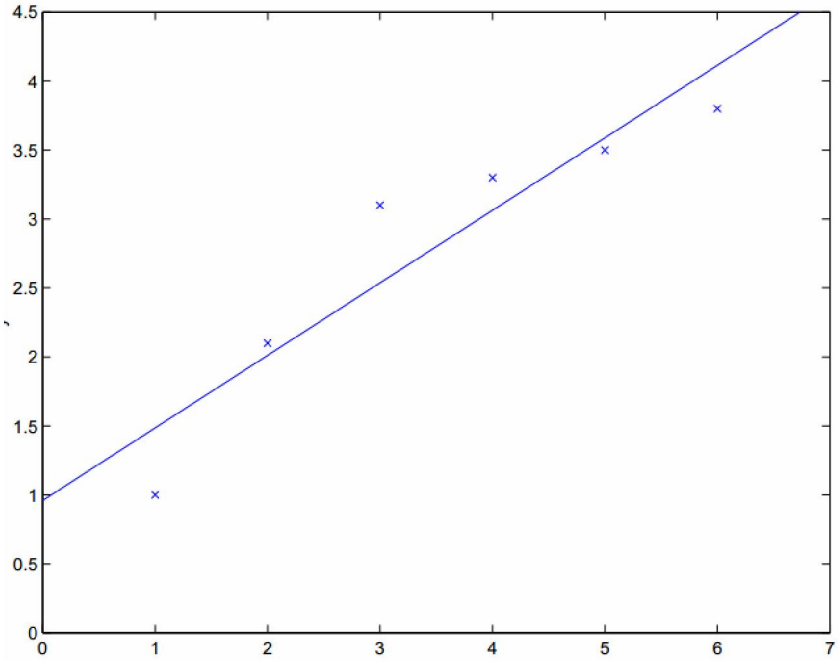


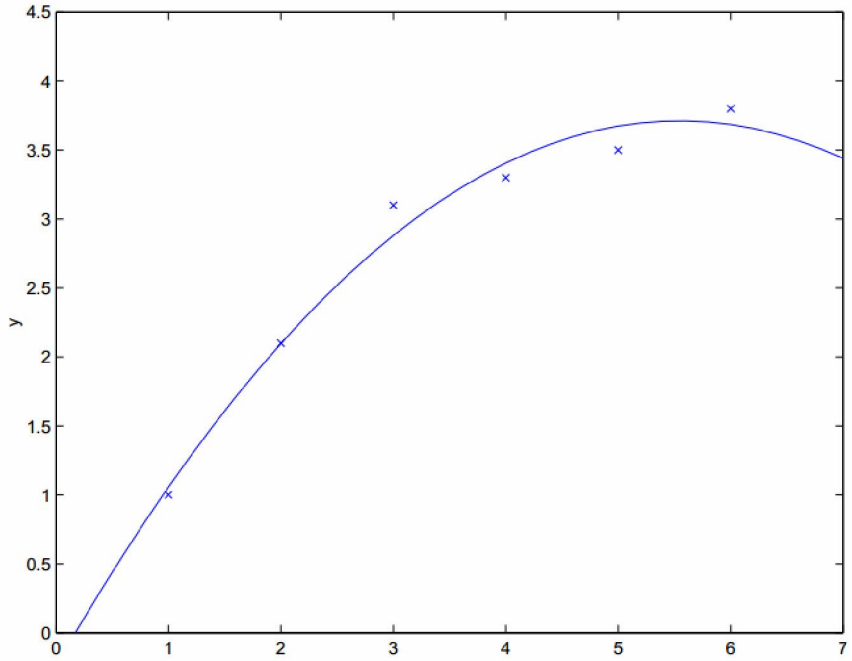




线性回归的进一步分析

- 可以对样本是非线性的，只要对参数 θ 线性

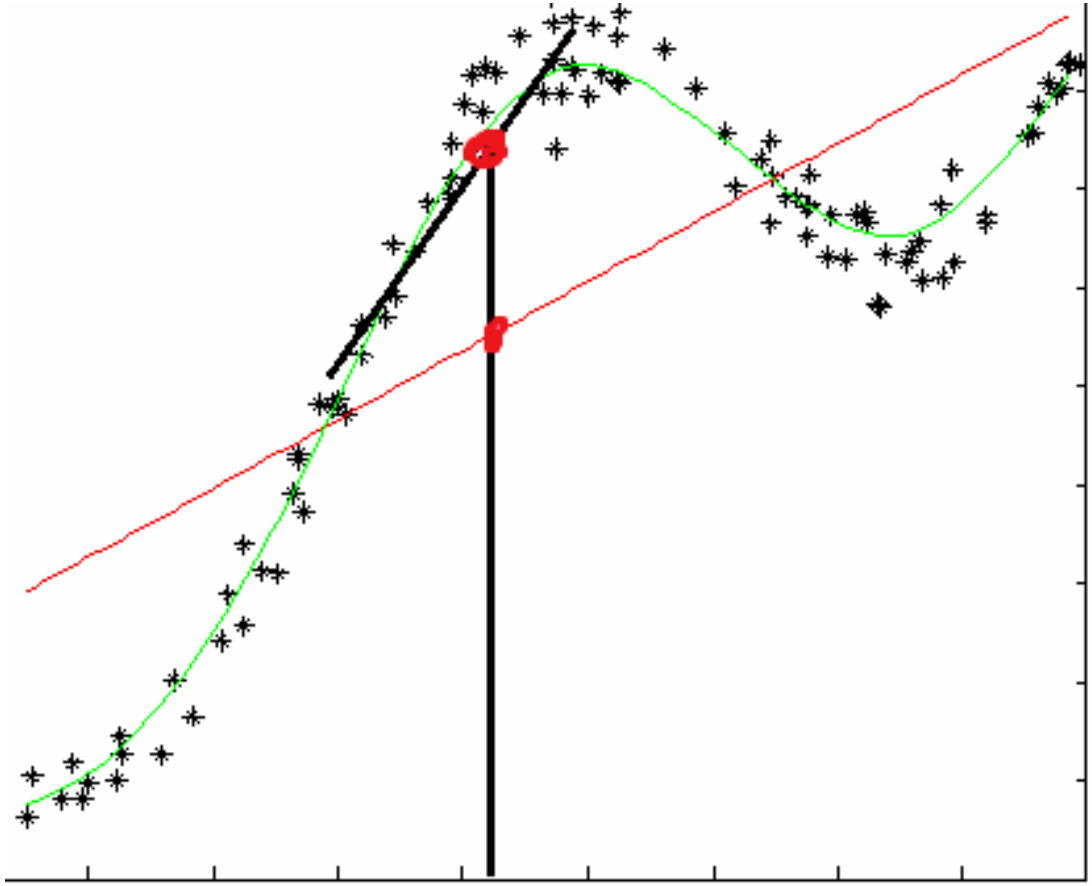




$$y = \theta_0 + \theta_1 x + \theta_2 x^2$$

局部加权回归

- 黑色是样本点
- 红色是线性回归曲线
- 绿色是局部加权回归曲线



局部加权线性回归

- LWR：Locally Weighted linear Regression

1. Fit θ to minimize $\sum_i (y^{(i)} - \theta^T x^{(i)})^2$
2. Output $\theta^T x$.

1. Fit θ to minimize $\sum_i w^{(i)} (y^{(i)} - \theta^T x^{(i)})^2$
2. Output $\theta^T x$.

权值的设置

- ω 的一种可能的选择方式(高斯核函数):

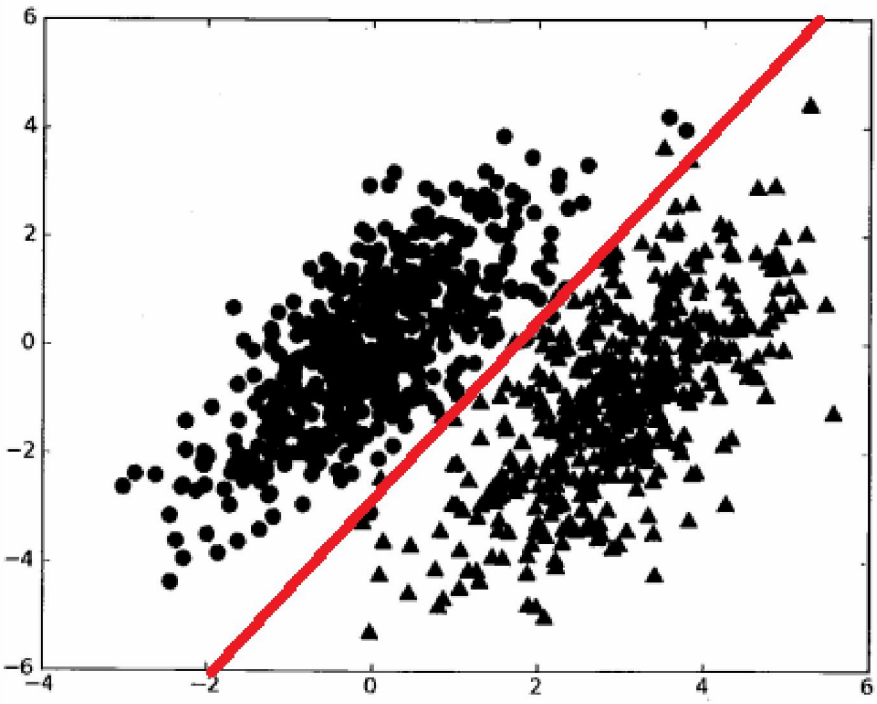
$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right)$$

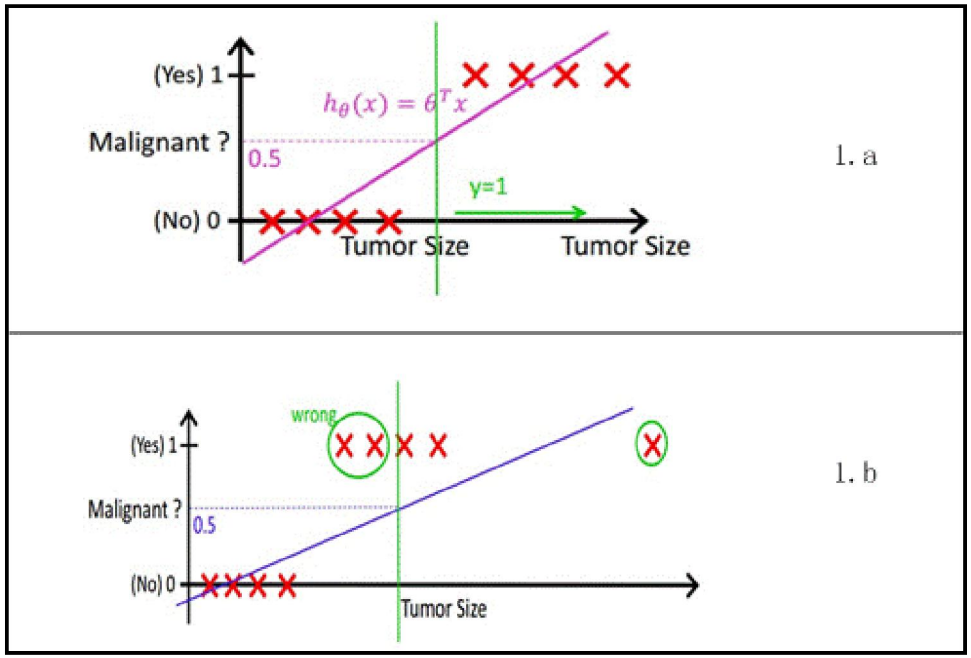
- τ 称为带宽，它控制着训练样本随着与$x^{(i)}$距离的衰减速率。
- 多项式核函数

$$\kappa(\boldsymbol{x}_1, \boldsymbol{x}_2) = (\langle \boldsymbol{x}_1, \boldsymbol{x}_2 \rangle + R)^d$$

  - 在SVM章节继续核函数的讨论。

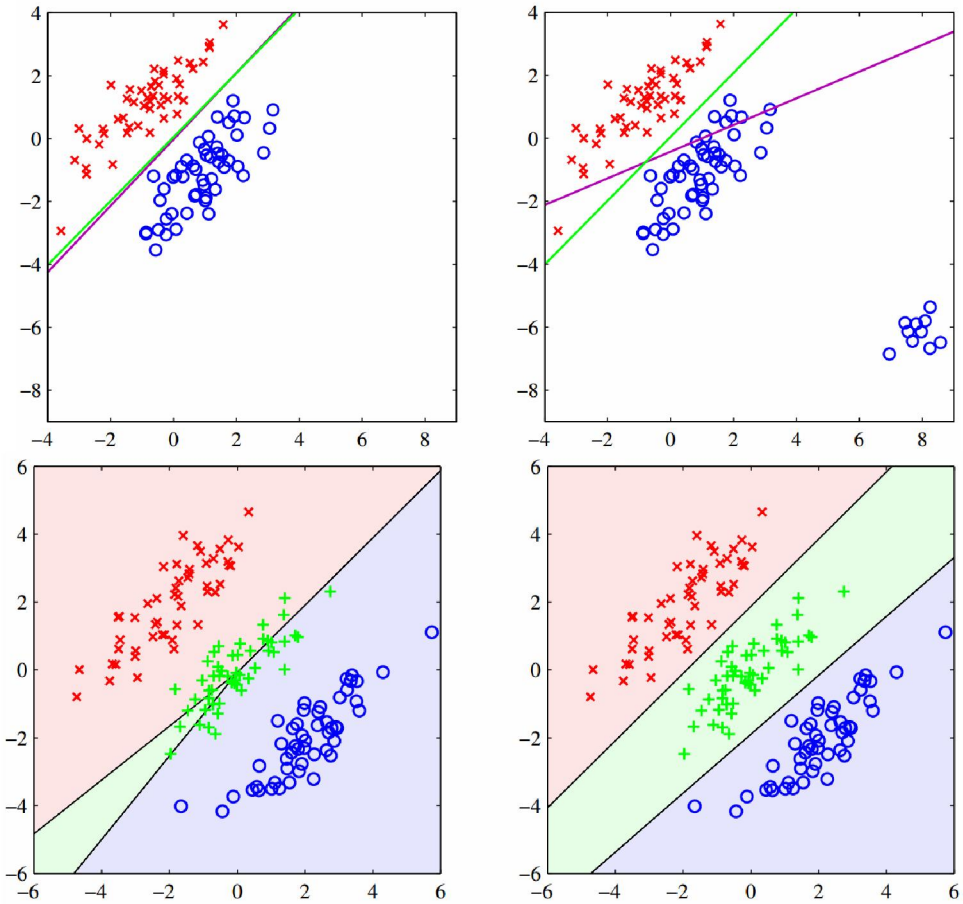
用回归解决分类问题，如何？





线性回归-Logistic回归

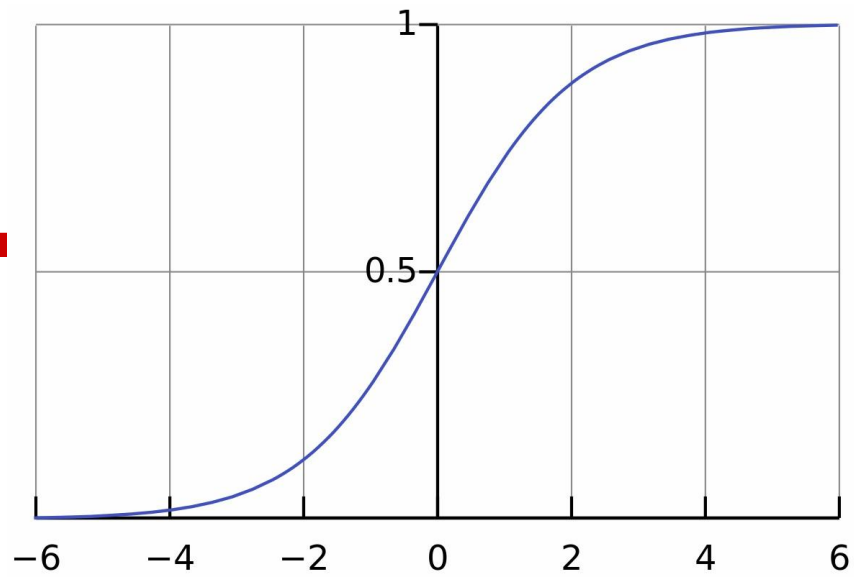
- 紫色：
  - 线性回归
- 绿色：
  - Logistic回归
- 左侧：
  - 线性回归
- 右侧：
  - Softmax回归



Logistic回归

- Logistic/sigmoid函数

$$h_\theta(x)=g(\theta^T x)=\frac{1}{1+e^{-\theta^T x}}$$



$$g(z)=\frac{1}{1+e^{-z}}$$

$$g'(x)=\left(\frac{1}{1+e^{-x}}\right)'=\frac{e^{-x}}{\left(1+e^{-x}\right)^2}$$

$$=\frac{1}{1+e^{-x}}\cdot\frac{e^{-x}}{1+e^{-x}}=\frac{1}{1+e^{-x}}\cdot\left(1-\frac{1}{1+e^{-x}}\right)$$

$$=g(x)\cdot(1-g(x))$$

Logistic回归参数估计

- 假定：

$$
\begin{aligned}
P(y=1 \mid x;\theta) &= h_\theta(x) \\
P(y=0 \mid x;\theta) &= 1-h_\theta(x)
\end{aligned}
$$

$$
p(y \mid x;\theta) = (h_\theta(x))^y\,(1-h_\theta(x))^{1-y}
$$

$$
\begin{aligned}
L(\theta) &= p(\vec{y} \mid X;\theta) \\
&= \prod_{i=1}^{m} p(y^{(i)} \mid x^{(i)};\theta) \\
&= \prod_{i=1}^{m} \left(h_\theta(x^{(i)})\right)^{y^{(i)}} \left(1-h_\theta(x^{(i)})\right)^{1-y^{(i)}}
\end{aligned}
$$

对数似然函数

$$
\begin{aligned}
\ell(\theta) &= \log L(\theta) \\
&= \sum_{i=1}^{m} y^{(i)} \log h(x^{(i)}) + (1 - y^{(i)}) \log(1 - h(x^{(i)}))
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial}{\partial \theta_j} \ell(\theta) &= \left(y \frac{1}{g(\theta^T x)} - (1-y) \frac{1}{1 - g(\theta^T x)} \right) \frac{\partial}{\partial \theta_j} g(\theta^T x) \\
&= \left(y \frac{1}{g(\theta^T x)} - (1-y) \frac{1}{1 - g(\theta^T x)} \right) g(\theta^T x)(1 - g(\theta^T x)) \frac{\partial}{\partial \theta_j} \theta^T x \\
&= \left(y(1 - g(\theta^T x)) - (1-y) g(\theta^T x) \right) x_j \\
&= (y - h_\theta(x))\, x_j
\end{aligned}
$$

参数的迭代

- Logistic回归参数的学习规则：

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

- 比较上面的结果和线性回归的结论的差别：
  - 它们具有相同的形式！

Repeat until convergence {

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

Loop {

for i=1 to m, {

$$\theta_j := \theta_j + \alpha \left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

}

}

对数线性模型

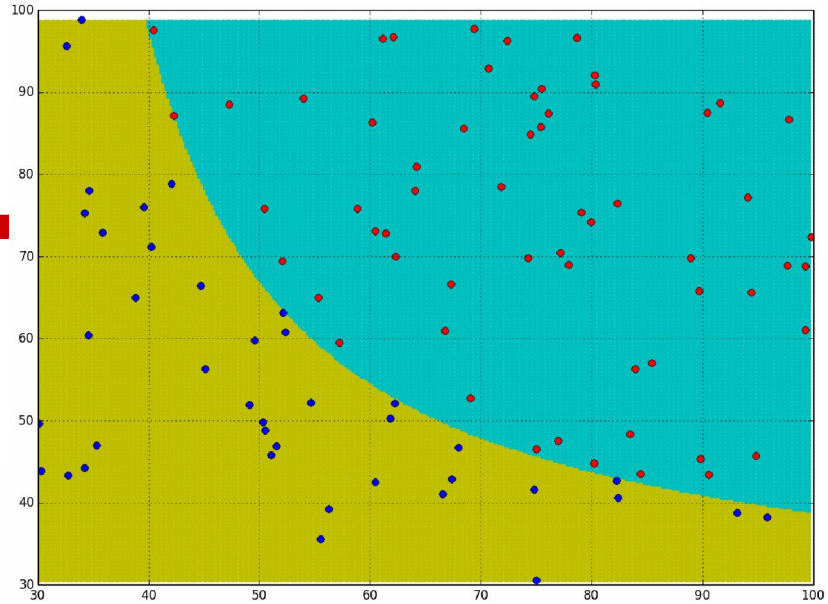
- 一个事件的几率odds，是指该事件发生的概率与该事件不发生的概率的比值。
- 对数几率：logit函数

$$
\begin{aligned}
P(y=1 \mid x;\theta) &= h_\theta(x) \\
P(y=0 \mid x;\theta) &= 1-h_\theta(x)
\end{aligned}
$$

$$
\log it(p)=\log\frac{p}{1-p}=\log\frac{h_\theta(x)}{1-h_\theta(x)}=\log\left(\frac{\frac{1}{1+e^{-\theta^T x}}}{\frac{e^{-\theta^T x}}{1+e^{-\theta^T x}}}\right)=\theta^T x
$$

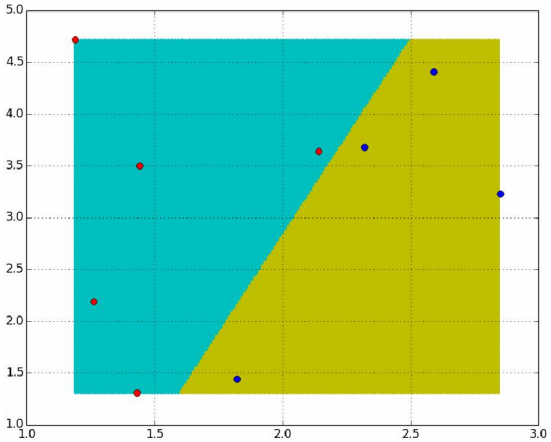
分类：Logistic回归

- 沿Logistic函数的负梯度
- 维度提升

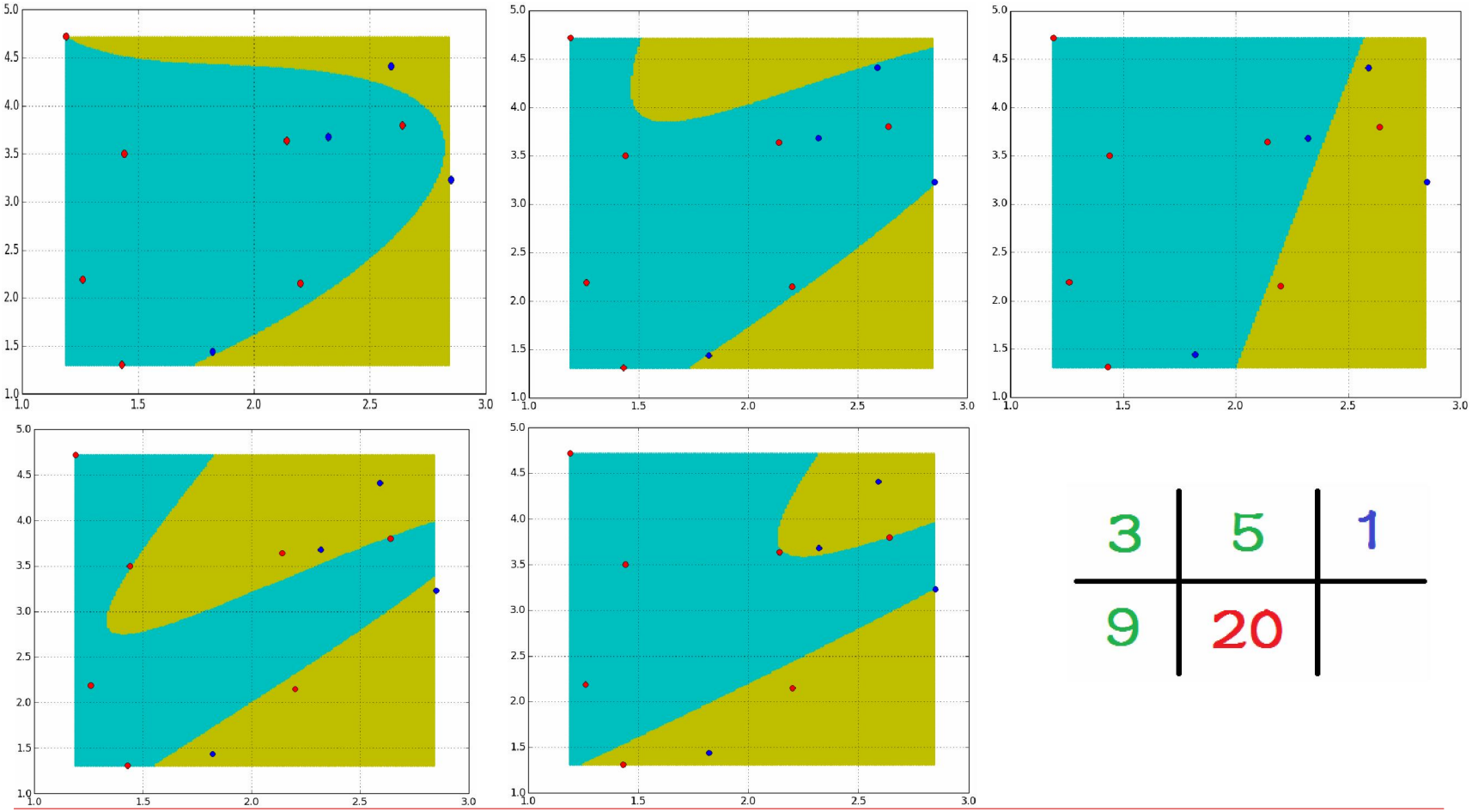



```
def logistic_regression(data, alpha, lamda):
    n = len(data[0]) - 1
    w = [0 for x in range(n)]
    w2 = [0 for x in range(n)]
    for times in range(10000):
        for d in data:
            for i in range(n):
                w2[i] = w[i] + alpha * (d[n] - h(w,d))*d[i] + lamda * w[i]
            for i in range(n):
                w[i] = w2[i]
            print w
    return w
```

```
def feature_whole(x1, x2, e):
    d = [1]
    for n in range(1,e+1):
        for i in range(n+1):
            d.append(pow(x1, n-i) * pow(x2, i))
    return d
```



数据升维：“选取特征”

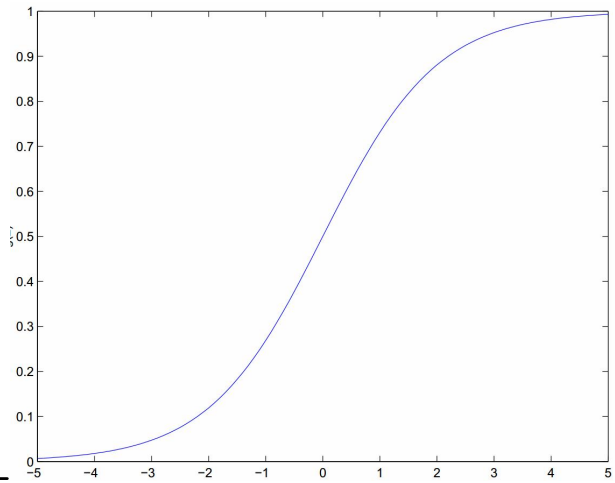


复习：指数族

- 指数族概念的目的，是为了说明广义线性模型Generalized Linear Models
- 凡是符合指数族分布的随机变量，都可以用GLM回归分析

广义线性模型GLM

- y不再只是正态分布，而是扩大为指数族中的任一分布；
- 变量x → g(x) → y
  - 连接函数g
- 连接函数g单调可导
  - 如Logistic回归中的 $g(z)=\dfrac{1}{1+e^{-z}}$
  - 拉伸变换：$g(z)=\dfrac{1}{1+e^{-\lambda z}}$



线性方程的最小二乘问题

- 原问题 $\quad \text{minimize} \quad x^T x, \quad x \in \mathbf{R}^n$

  $\text{subject to} \quad Ax = b$

- Lagrange函数

$$L(x,\nu) = x^T x + \nu^T (Ax - b)$$

- Lagrange对偶函数

$$g(\nu) = -\frac{1}{4}\nu^T A A^T \nu - b^T \nu$$

  - 对L求x的偏导，带入L，得到g
  - 对g求v的偏导，求g的极大值，作为原问题的最小值

求L的对偶函数 $L(x,\nu)=x^T x+\nu^T(Ax-b)$

$$\frac{\partial L}{\partial x}=\frac{\partial\left(x^T x+\nu^T(Ax-b)\right)}{\partial x}=2x+A^T\nu\overset{令}{=}0\Rightarrow x^*=-\frac{1}{2}A^T\nu$$

$$\begin{aligned}
&L(x,\nu)=x^T x+\nu^T(Ax-b)\\
&=\left(-\frac{1}{2}A^T\nu\right)^T\left(-\frac{1}{2}A^T\nu\right)+\nu^T\left(A\left(-\frac{1}{2}A^T\nu\right)-b\right)\\
&=\frac{1}{4}\nu^T AA^T\nu-\frac{1}{2}\nu^T AA^T\nu-\nu^T b\\
&=-\frac{1}{4}\nu^T AA^T\nu-\nu^T b\\
&\overset{\Delta}{=}g(\nu)
\end{aligned}$$

求对偶函数的极大值 $g(\nu)=-\frac{1}{4}\nu^T AA^T\nu-\nu^T b$

$$\frac{\partial g}{\partial \nu}=\frac{\partial\left(-\frac{1}{4}\nu^T AA^T\nu-\nu^T b\right)}{\partial \nu}=-\frac{1}{2}AA^T\nu-b\overset{令}{=}0$$

$$\Rightarrow AA^T\nu=-2b$$

$$\Rightarrow A^T AA^T\nu=-2A^T b$$

$$\Rightarrow A^T\nu=-2\left(A^T A\right)^{-1}A^T b$$

$$\Rightarrow -\frac{1}{2}A^T\nu=\left(A^T A\right)^{-1}A^T b$$

$$\Rightarrow x^*=\left(A^T A\right)^{-1}A^T b$$

极小值点 $x^* = \left(A^T A\right)^{-1} A^T b$

- 极小值：$\min\left(x^T x\right)$

$$= \left(\left(A^T A\right)^{-1} A^T b\right)^T \left(\left(A^T A\right)^{-1} A^T b\right)$$

$$= b^T A\left(A^T A\right)^{-1}\left(A^T A\right)^{-1} A^T b$$

$$= b^T A\left(A^T A\right)^{-2} A^T b$$

- 极小值点的结论，和通过线性回归计算得到的结论是完全一致的。
  - 线性回归问题具有强对偶性。

参考文献

- Prof. Andrew Ng, Machine Learning, Stanford University
- 李航，统计学习方法，清华大学出版社，2012
- Convex Optimization, Stephen Boyd,Lieven Vandenberghe, Cambridge University Press, 2004
  - 中译本：王书宁，许鋆，黄晓霖 译，凸优化，清华大学出版社，2013

我们在这里

- 问题答疑：http://www.xxwenda.com/
  - 烦请邀请visio或其他人回答问题

- 本课程QQ群：
  - 471632108(小象学院：机器学习群)

感谢大家！

恳请大家批评指正！